Görev ve Proje Yönetim Sistemi API Geliştirme Projesi

1. Proje Özeti, Amacı ve Senaryo

1.1 Proje Özeti ve Amacı

Bu proje, Node.js and Express.js kullanılarak geliştirilen; ekiplerin görev dağılımlarını, proje süreçlerini ve performanslarını yönetebilecekleri kapsamlı bir RESTful API sunmaktadır. Kurum içi iletişimi ve görev takibini dijitalleştirerek verimliliği artırmak ve yöneticilere detaylı raporlar sunabilmek temel amaçtır. Veriler yerel JSON dosyalarında saklanmaktadır.

1.2 Senaryo

Kurumsal bir ekip veya yazılım ajansı, günlük operasyonlarındaki görev atamalarını, durum değişikliklerini ve çalışan performansını güvenli, merkezi bir sistemden uçtan uca yönetip raporlamak istemektedir.

2. Proje Mimarisi ve Detaylı Dosya Yapısı

```
proje-klasoru/
├── data/
│   ├── logs/
│   │   └── log.json              # Uygulama log kayıtlarının tutulduğu dosya
│   ├── gorevler.json             # Görev verilerinin saklandığı JSON deposu
│   └── calisanlar.json           # Çalışan verilerinin saklandığı JSON deposu
├── src/
│   ├── middleware/               # Ara yazılımların (Middleware) bulunduğu klasör
│   │   ├── logger.js             # İstekleri ve hataları loglayan ara yazılım
│   │   └── validateGorev.js      # Görev verilerini doğrulayan validasyon ara
yazılımı
│   ├── route/                    # REST API rotalarının (Router) bulunduğu klasör
│   │   ├── calisan.js            # Çalışan işlemleri için uç noktalar
│   │   ├── gorev.js              # Görev işlemleri için uç noktalar
│   │   └── rapor.js              # Raporlama işlemleri için uç noktalar
│   └── utils/                    # Yardımcı modüllerin bulunduğu klasör
│       └── dosya.js              # Dosya okuma/yazma (I/O) işlemlerini yöneten
modül
├── app.js                        # Express uygulamasının yapılandırıldığı dosya
├── server.js                     # Sunucuyu (port) ayağa kaldıran ana giriş
noktası
├── package-lock.json             # Bağımlılıkların kilit versiyonlarını tutan
dosya
└── package.json                  # Proje bağımlılıkları ve yapılandırma dosyası
```

3. İstek Yönetimi, Doğrulama ve Test

Sistem stabilitesini, veri bütünlüğünü ve izlenebilirliği sağlamak adına projede profesyonel ara yazılımlar ve test stratejileri benimsenmiştir.

• İstek Loglama ve Takibi (logger.js): Sisteme yapılan tüm endpoint çağrıları ve işlemler 'logger.js' ara yazılımı vasıtasıyla eşzamanlı olarak loglanmakta ve geriye dönük inceleme için log.json dosyasına kaydedilmektedir.

• Veri Doğrulama (validateGorev.js): Görev oluşturma ve güncelleme süreçlerinde veri bütünlüğünü ve şema uyumluluğunu garanti altına almak amacıyla 'validateGorev.js' doğrulama ara yazılımı kullanılmaktadır.
• Test Stratejisi ve Doğrulama: Geliştirilen tüm uç noktaların işlevselliği, Postman test senaryoları aracılığıyla doğrulanmaktadır.

4. Uç Nokta (Endpoint) Tabloları

'/gorev' Route

Method	URL	Body	İşlev / Yanıt
POST	/gorev	{gorev_adi, gorev_detayi, durum, oncelik, calisan_id}	Yeni görev oluşturur (validateGorev ile) / 201 Created
GET	/gorev	-	Tüm görevleri listeler (Durum, öncelik, sıralama, sayfalama) / 200 OK
GET	/gorev/ara	- (Query: ?key=...)	Görev adı veya detayı içinde kelime bazlı arama yapar / 200 OK
GET	/gorev/:id	-	ID'ye göre tekil görev detayını getirir / 200 OK
PUT	/gorev/:id	{gorev_adi, gorev_detayi, durum, oncelik}	Görev detaylarını günceller (Validation kurallarına tabidir) / 200 OK
PATCH	/gorev/:id/ata	{calisan_id}	Göreve geçerli bir çalışan atar / 200 OK
DELETE	/gorev/:id	-	Belirtilen ID'ye sahip görevi sistemden siler / 200 OK

'/calisan' Route

Method	URL	Body	İşlev / Yanıt
POST	/calisan	{ad_soyad, email, departman, unvan}	Otomatik ID (101+) atayarak yeni çalışan kaydeder / 201 Created
GET	/calisan	-	Sistemdeki tüm çalışanları listeler / 200 OK
GET	/calisan/:id/gorevler	-	Belirli bir çalışanın üzerine atanmış tüm görevleri listeler / 200 OK
DELETE	/calisan/:id	-	Çalışanı siler ve üzerine kayıtlı görevlerin calisan_id değerini null yapar / 200 OK

'/rapor' Route

Method	URL	Body	İşlev / Yanıt
GET	/rapor/completed	-	Durumu 'tamamlandı' olan görevlerin sayısını döner / 200 OK
GET	/rapor/pending	-	Durumu 'beklemede' olan görevlerin sayısını döner / 200 OK
GET	/rapor/ozet	-	Toplam, tamamlanan, bekleyen ve devam eden görev sayılarını özetler / 200 OK
GET	/rapor/log	-	log.json dosyasındaki sistem log geçmişini listeler / 200 OK

5. Veri Yapıları

API üzerinden dönen JSON formatlı modellerin şeması ve beklenen veri tipleri aşağıda gösterilmiştir:

```
[Görev JSON]
{
  "id": "Integer",
  "gorev_adi": "String",
  "gorev_detayi": "String",
  "durum": "String (Örn: 'tamamlandı', 'beklemede', 'devam ediyor')",
  "oncelik": "String (Örn: 'düşük', 'orta', 'yüksek')",
  "calisan_id": "Integer / null"
}

[Çalışan JSON]
{
  "id": "Integer",
  "ad_soyad": "String",
  "email": "String",
  "departman": "String",
  "unvan": "String"
}

[Log JSON]
{
  "zaman": "String (ISO 8601)",
  "metod": "String",
  "url": "String",
  "ip": "String"
}
```

6. Kullanılan Teknolojiler

• Backend (Arka Uç): Node.js ve Express.js (Asenkron API geliştirme)
• Veri Depolama: Yerel Dosya Sistemi (File System - fs modülü ile JSON tabanlı veri saklama)
• Test ve Dokümantasyon: Postman (API uç noktalarının test edilmesi ve doğrulanması)

7. Bağlantılar

GitHub Repo Linki: https://github.com/KayliAli/Tnc-BackendExpress
Postman Test Ekran Görüntüleri: https://github.com/KayliAli/Tnc-BackendExpress/blob/main/docs/Detayli_API_Test_Raporu.pdf